

2.1. Лабораторная работа №1 – Вещественные типы данных	8
2.2. Лабораторная работа №2 – Циклы	11
2.3. Лабораторная работа №3 – Одномерные массивы	15
2.4. Лабораторная работа №4 – Сортировка массива	22
2.5. Лабораторная работа №5 – Структуры.....	27
2.6. Лабораторная работа №6 – Строки и файлы	37
2.7. Лабораторная работа №7 – Приложение ОС Windows с выводом в окно текста и фигур	50
2.8. Лабораторная работа №8 – Средства ввода/вывода в ОС Windows.....	59
2.9. Лабораторная работа №9 – Преобразование растровых изображений	70
2.10. Лабораторная работа №10 – Построение графика функции.....	75

Лабораторная №1

$$18 \quad \left| \frac{(a-b)^3 - (a^3)}{b^3 - 3ab^2 - 3a^2b}, \text{ при } a=1000, b=0.0001 \right.$$

```

main.cpp +
1  #include <iostream>
2  #include <cmath> // Библиотека математических функций
3
4  using namespace std; // директива, которая говорит компилятору использовать пространство имен std
5
6  // Вариант 18 Парменова Дарья
7  int main() {
8      double a = 1000; // задаем переменные
9      double b = 0.0001;
10     double res = (pow((a-b), 3) - pow(a, 3))/(pow(b, 3) - 3*a*b*b - 3*a*a*b);
11
12     std::cout << "(a-b)^3 - (a^3) / b^3 - 3ab^2 - 3a^2b = " << res << std::endl; // вывод результата
13
14     return 0;
15 }

```

$$(a-b)^3 - (a^3) / b^3 - 3ab^2 - 3a^2b = 1$$

Лабораторная №2

$$12) \quad a_n = \frac{2^n n}{n^n}.$$

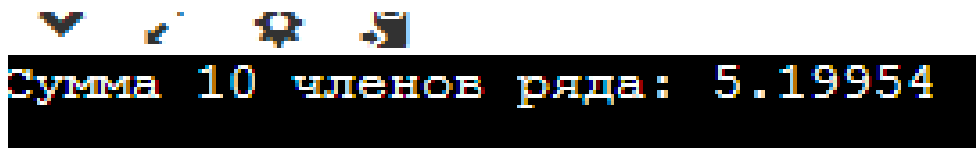
```
#include <iostream>
#include <cmath>

using namespace std; // директива, которая говорит компилятору использовать пространство имен std

int main() {
    double sum = 0; // задаем переменную суммы

    for (int n = 1; n <= 10; n++) { // цикл от 1 до n, где с каждым шагом n увеличивается на 1
        double a = (pow(2, n) * n) / pow(n, n); // формула варианта
        sum += a;
    }

    cout << "Сумма 10 членов ряда: " << sum << endl; // вывод результата
}
```



Сумма 10 членов ряда: 5.19954

Лабораторная №3

12.

- Сформировать одномерный массив целых чисел, используя генератор случайных чисел. Длина массива задается пользователем.
- Вывести на экран монитора полученный массив.
- Поменять местами минимальный и максимальный элементы массива.
- Удалить из массива все элементы, превышающие его среднее значение более, чем на 10%.
- Вывести на экран монитора полученный массив.

```
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

void printArray(int arr[], int size) { // Функция для вывода массива, void - функция, которая ничего не возвращает
    for (int i = 0; i < size; i++) { // через цикл выводим каждый элемент массива
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    // пользователь задает длину массива
    int size;
    cout << "Введите длину массива: ";
    cin >> size;

    int* arr = new int[size]; // создаем массив нужной длины

    for (int i = 0; i < size; i++) { // через цикл и метод rand добавляем туда числа в диапазоне от 1 до 100
        arr[i] = rand() % 100 + 1;
    }
}
```

```

// Выводим массив
cout << "Массив: ";
printArray(arr, size);

int minIndex = 0;
int maxIndex = 0;

// Цикл, который находит максимальный и минимальный элемент массива
for (int i = 1; i < size; i++) {
    if (arr[i] < arr[minIndex]) {
        minIndex = i;
    }
    if (arr[i] > arr[maxIndex]) {
        maxIndex = i;
    }
}

// Меняем их местами
int temp = arr[minIndex];
arr[minIndex] = arr[maxIndex];
arr[maxIndex] = temp;

// Поиск среднего значения массива
int sum = 0;
for (int i = 0; i < size; i++) {
    sum += arr[i];
}
double average = static_cast<double>(sum) / size;

```

```

// Удаление из массива элементов, через 2 цикла
int newSize = size;
for (int i = 0; i < newSize; i++) {
    if (arr[i] > average * 1.1) {
        for (int j = i; j < newSize - 1; j++) {
            arr[j] = arr[j + 1];
            newSize--;
            i--;
        }
    }
}
// Выводим конечный массив
cout << "Массив после удаления элементов: ";
printArray(arr, newSize);

return 0;
}

```

```

Введите длину массива: 5
Массив: 32 61 17 70 4
Массив после удаления элементов: 32 17 4

```

Лабораторная №4

```
#include <iostream>
#include <algorithm>

using namespace std;

void printArray(int arr[], int size) { // Функция для вывода массива через цикл
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {

    int size;
    cout << "Введите размер массива: "; // Ввод пользователем размера массива
    cin >> size;

    int arr[size]; // Генерируем случайные элементы в диапазоне от - 100 до 100
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 200 - 100;
    }

    int a, b; // Ввод пользователем интервала
    cout << "Введите интервал [a, b]: ";
    cin >> a >> b;

    int count = 0; // Подсчет кол-ва элементов в интервале через цикл,
    for (int i = 0; i < size; i++) { // проходящий по всем элементам с условием
        if (arr[i] >= a && arr[i] <= b) {
            count++;
        }
    }
}
```

```

cout << "Массив: ";           // Вывод массива, а затем кол-ва элементов в интервале
printArray(arr, size);

cout << "Количество элементов в интервале [" << a << ", " << b << "]: " << count << endl;

int maxIndex = 0;             // Поиск максимального элемента через цикл с условием
for (int i = 1; i < size; ++i) {
    if (arr[i] > arr[maxIndex]) {
        maxIndex = i;
    }
}

int sum = 0;                  // Подсчет суммы элементов после максимального элемента
for (int i = maxIndex + 1; i < size; ++i) {
    sum += arr[i];
}
cout << "Сумма элементов массива после максимального элемента: " << sum << endl;

for (int i = 0; i < size-1; ++i) { // Сортировка пузырьком
    for (int j = 0; j < size-i-1; ++j) {
        if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

cout << "Отсортированный массив: ";
printArray(arr, size);
}

```

Введите размер массива: 10

Введите интервал [a, b]: 10 50

Массив: 83 -14 77 15 93 35 86 -8 -51 -79

Количество элементов в интервале [10, 50]: 2

Сумма элементов массива после максимального элемента: -17

Отсортированный массив: -79 -51 -14 -8 15 35 77 83 86 93

Лабораторная №5

```
#include <iostream>
#include <string>

using namespace std;

struct NOTE { // ОБЪЯВЛЕНИЕ СТРУКТУРЫ, СОСТОЯЩЕЙ ИЗ 3 СТРОК И ОДНОГО МАССИВА С РАЗМЕРОМ 3(ДЕНЬ МЕСЯЦ ГОД)
    string firstName;
    string lastName;
    string phoneNumber;
    int birth[3];
};

void sortNotes(NOTE notes[], int n) { // ФУНКЦИЯ ДЛЯ СОРТИРОВКИ ПО АЛФАВИТУ
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (notes[j].lastName > notes[j + 1].lastName) {
                NOTE temp = notes[j];
                notes[j] = notes[j + 1];
                notes[j + 1] = temp;
            }
        }
    }
}

int main(void) { // ЗАДАЕМ РАЗМЕР СТРУКТУРЫ
    const int size = 8;
    NOTE notes[size];

    cout << "Введите данные для " << size << " записей:\n"; // ВВОД ВСЕХ ДАННЫХ ЧЕРЕЗ ЦИКЛ
    for (int i = 0; i < size; ++i) {
        cout << "Фамилия: ";
        cin >> notes[i].lastName;
        cout << "Имя: ";
        cin >> notes[i].firstName;
        cout << "Номер телефона: ";
        cin >> notes[i].phoneNumber;
        cout << "День рождения: ";
        for (int j = 0; j < 3; ++j) {
            cin >> notes[i].birth[j];
        }
    }

    sortNotes(notes, size); // СОРТИРОВКА
```

```
    cout << "Отсортированные записи:\n";
    for (int i = 0; i < size; ++i) { // ВЫВОД СТРУКТУРЫ
        cout << notes[i].lastName << " " << notes[i].firstName << " " << notes[i].phoneNumber << " ";
        for (int j = 0; j < 3; ++j) {
            cout << notes[i].birth[j] << " ";
        }
        cout << endl;
    }

    int month;
    cout << "Введите номер месяца для поиска: "; // ВВОД НУЖНОГО МЕСЯЦА
    cin >> month;

    bool found = false;
    for (int i = 0; i < size; ++i) { // ПОИСК НУЖНОГО ЧЕЛОВЕКА В МАССИВЕ СТРУКТУР
        if (notes[i].birth[1] == month) {
            cout << "Фамилия: " << notes[i].lastName << "\n";
            cout << "Имя: " << notes[i].firstName << "\n";
            cout << "Номер телефона: " << notes[i].phoneNumber << "\n";
            cout << "День рождения: " << notes[i].birth[0] << "/" << notes[i].birth[1] << "/" << notes[i].birth[2] << "\n\n";
            found = true;
        }
    }

    if (!found) {
        std::cout << "Людей с днем рождения в указанном месяце нет.\n"; // ВЫВОД СООБЩЕНИЯ ЕСЛИ ТАКИХ ЛЮДЕЙ НЕТ
    }

    return 0;
}
```

Введите данные для 8 записей:

фамилия: Парменова

Имя: Дарья

Номер телефона: 111

День рождения: 27 09 2002

фамилия: Лебедев

Имя: Номер телефо^а: Иван

День рождения: 1 01 2001

фамилия: ф

Имя: ф

Номер телефона: 1

День рождения: 1 1 1

фамилия: Й

Имя: Й

Номер телефона: 1

День рождения: 1 2 3

фамилия: В

Имя: В

Номер телефона: 1

День рождения: 1 1 1

фамилия: А

Имя: А

Номер телефона: 4

День рождения: 1 2 3

фамилия: Н

Имя: Н

Номер телефона: 3

День рождения: 1 2 3

фамилия: Л

Имя: Л

Номер телефона: 5

День рождения: 6 7 8

Отсортированные ^аписи:

А А 4 1 2 3

В В 1 1 1 1

Й Й 1 1 2 3

Л Л 5 6 7 8

Лебедев И Иван 1 1 2001

Н Н 3 1 2 3

Паорменова Дарья 111 27 9 2002

ф ф 1 1 1 1

Введите номер месяца для поиска: 09

фамилия: Паорменова

Имя: Дарья

Номер телефона: 111

День рождения: 27/9/2002

Лабораторная №6

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  using namespace std;
6
7  // Функция для вывода текста на экран
8  void printText(const string& text) {
9      bool newSentence = true; // Флаг для определения начала нового предложения
10
11      for (char c : text) {
12          if (c == '.') {
13              cout << c << endl; // Выводим точку и переходим на новую строку
14              newSentence = true; // Следующий символ начинает новое предложение
15          }
16          else if (c != ' ' || !newSentence) {
17              cout << c; // Выводим символ, если это не лишний пробел и не начало нового предложения
18              newSentence = false; // Предложение продолжается после пробела
19          }
20      }
21  }
22
23 // Функция для вывода текста на экран, убирая лишние пробелы и начиная каждое предложение с новой строки
24 void printFormattedText(const string& text) {
25     bool newSentence = true; // Флаг для определения начала нового предложения
26     bool prevSpace = false; // Флаг для определения, был ли предыдущий символ пробелом
27
28     for (char c : text) {
29         if (c == '.') {
30             cout << c << endl; // Выводим точку и переходим на новую строку
31             newSentence = true; // Следующий символ начинает новое предложение
32             prevSpace = false; // После точки пробелы не нужны
33         }
34         else if (c == ' ') {
35             if (!prevSpace && !newSentence) {
36                 cout << c; // Выводим пробел, если это не начало предложения и не лишний пробел
37             }
38             prevSpace = true; // Запоминаем, что предыдущий символ был пробелом
39         }
40         else {
41             cout << c; // Выводим символ, если это не начало предложения и не пробел
42             newSentence = false; // Предложение продолжается после символа
43             prevSpace = false; // Предыдущий символ не пробел
44         }
45     }
46 }
47
48 int main() {
49     ifstream file("lab6.dat", ios::binary); // Открываем файл в бинарном режиме
50     if (!file) {
51         cerr << "Ошибка открытия файла!" << endl;
52         return 1;
53     }
54
55     // Получаем размер файла
56     file.seekg(0, ios::end);
57     size_t fileSize = file.tellg();
58     file.seekg(0, ios::beg);
59
60     // Выделяем достаточное количество памяти для чтения содержимого файла
61     string text;
62     text.resize(fileSize);
63
64     // Читаем содержимое файла в строку
65     file.read(&text[0], fileSize);
66
67     // Вывод содержимого файла с учетом пробелов
68     cout << "Text:" << endl;
69     printText(text);
70
71     cout << "Text without extra spaces and with a new line after each sentence:" << endl;
72     printFormattedText(text); // Выводим текст с форматированием
73
74     file.close(); // Закрываем файл
75
76     return 0;
77 }
```

Text:

The sun was shining brightly in the clear blue sky.

Birds chirped merrily as they flitted from tree to tree.

A gentle breeze rustled the leaves, creating a soothing melody.

Children laughed and played in the nearby park, their joy contagious.

It was a perfect day for a picnic by the shimmering lake.

Families gathered together, sharing food, stories, and laughter.

Memories were made that would last a lifetime.

Text without extra spaces and with a new line after each sentence:

The sun was shining brightly in the clear blue sky.

Birds chirped merrily as they flitted from tree to tree.

A gentle breeze rustled the leaves, creating a soothing melody.

Children laughed and played in the nearby park, their joy contagious.

It was a perfect day for a picnic by the shimmering lake.

Лабораторная №7

18. В окно светло-бирюзового цвета выведите круг диаметром 400 пикселей, разделенный пополам диагональю, проходящей вправо и вверх. Левую половину круга закрасьте красным цветом, правую – синим. Полукруги рисуются с помощью функций `Pie()` или `Chord()` класса `TCanvas`. Выведите поверх изображения свою фамилию шрифтом Times размером 80 пикселей, широким (шириной 80 пикселей), курсивом, черного цвета. Надпись расположите горизонтально.

```

#include <vcl.h>
#pragma hdrstop
#include "Unit1.h"

#pragma package(smart_init)
#pragma resource "*.dfm"

TForm1 *Form1;

//-----
__fastcall TForm1::TForm1(TComponent* Owner)
: TForm(Owner)
{
    this->OnPaint = DrawContent;
}

//-----
void __fastcall TForm1::DrawContent(TObject *Sender)
{
    Form1->Color = TColor(RGB(64, 224, 208)); // Фон окна

    int diameter = 400; // Задаем диаметр круга и координаты
    int xCenter = ClientWidth / 2;
    int yCenter = ClientHeight / 2;
    int radius = diameter / 2;

    // Угол поворота в радианах для синего полукруга
    double angle = 45 * M_PI / 180;

    // Вычисляем координаты для начальной точки дуги для синего полукруга
    int startX_blue = xCenter - radius;
    int startY_blue = yCenter + radius;

    // Вычисляем координаты для конечной точки дуги для синего полукруга
    int endX_blue = xCenter + radius * cos(angle);
    int endY_blue = yCenter - radius * sin(angle);

    // Рисуем полукруг синий
    Canvas->Brush->Color = clBlue;
    Canvas->Pie(xCenter - radius, yCenter - radius, xCenter + radius, yCenter + radius,
        startX_blue, startY_blue, endX_blue, endY_blue);

```

```

// Для начальных точек красного полукруга
double angle3 = (45 + 90) * M_PI / 180;
// Для конечных точек
double angle2 = (45 + 90) * M_PI / 180;

// Вычисляем новые координаты для начальной точки дуги для красного полукруга
int startX_red = xCenter - radius * cos(angle3);
int startY_red = yCenter - radius * sin(angle3);

// Вычисляем новые координаты для конечной точки дуги для красного полукруга
int endX_red = xCenter + radius * cos(angle2);
int endY_red = yCenter + radius * sin(angle2);

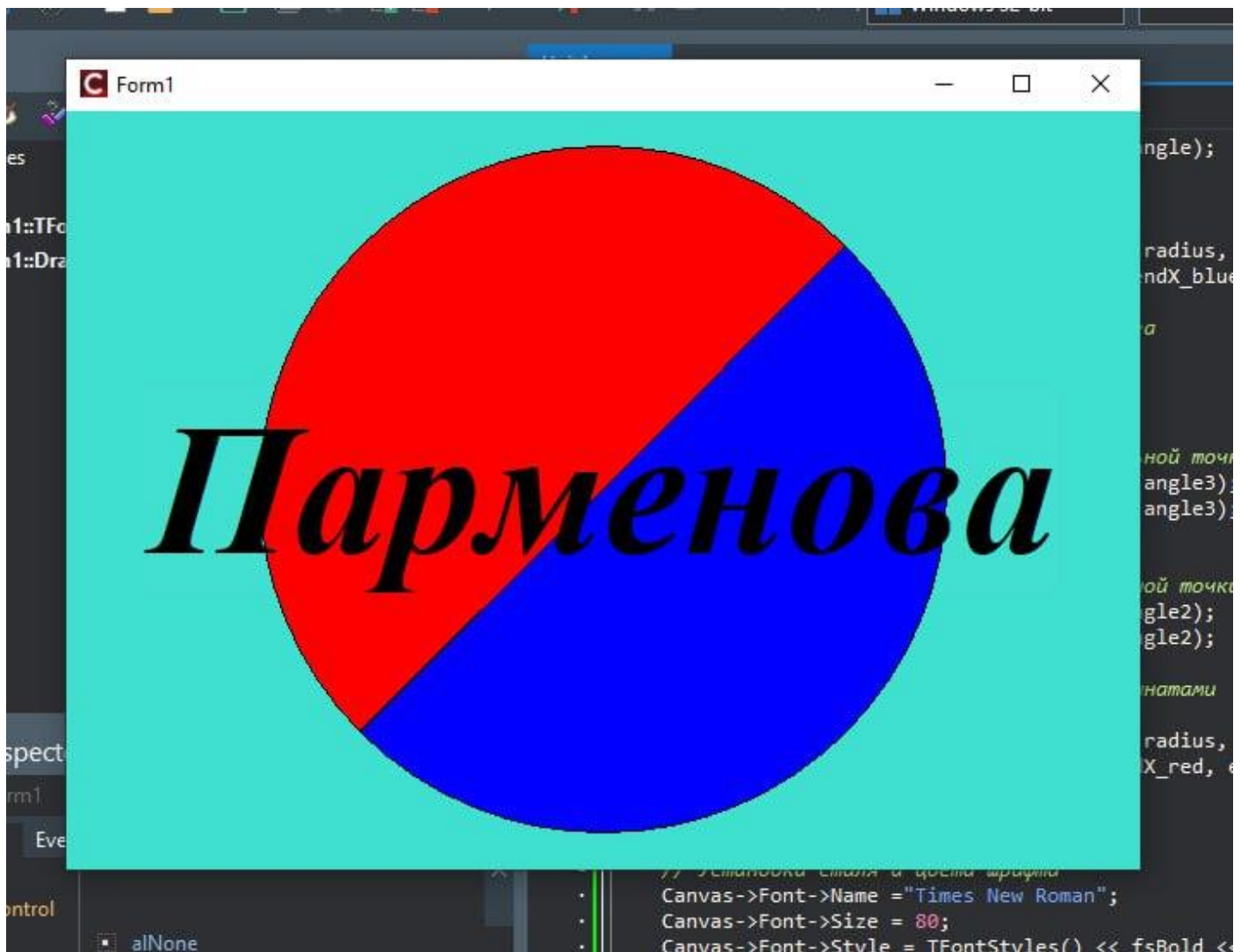
// Рисуем синий полукруг с новыми координатами
Canvas->Brush->Color = clRed;
Canvas->Pie(xCenter - radius, yCenter - radius, xCenter + radius, yCenter + radius,
    startX_red, startY_red, endX_red, endY_red);

// Установка стиля и цвета шрифта
Canvas->Font->Name = "Times New Roman";
Canvas->Font->Size = 80;
Canvas->Font->Style = TFontStyles() << fsBold << fsItalic;
Canvas->Font->Color = clBlack;

// Определяем центр окна для текста
AnsiString text = "Парменова";
int textWidth = Canvas->TextWidth(text);
int textHeight = Canvas->TextHeight(text);

// Выводим текст
Canvas->Brush->Style = bsClear; // Установка стиля кисти на прозрачный
Canvas->TextOut((ClientWidth - textWidth) / 2, (ClientHeight - textHeight) / 2, text);
}

```



Лабораторная №8

```
main.cpp  data.txt  ⋮
1  #include <iostream>
2  #include <fstream>
3  #include <cmath>
4
5  int main() {
6      // Ввод двух чисел с клавиатуры
7      double num1, num2;
8      std::cout << "Введите первое число: ";
9      std::cin >> num1;
10     std::cout << "Введите второе число: ";
11     std::cin >> num2;
12
13     // Запись чисел в файл
14     std::ofstream outFile("data.txt");
15     if (outFile.is_open()) {
16         outFile << num1 << " " << num2;
17         outFile.close();
18         std::cout << "Данные успешно сохранены в файл." << std::endl;
19     } else {
20         std::cout << "Ошибка при сохранении данных в файл." << std::endl;
21         return 1;
22     }
23 }
```

```
main.cpp  data.txt  ⋮
22      }
23
24      // Чтение чисел из файла
25      std::ifstream inFile("data.txt");
26      if (inFile.is_open()) {
27          double readNum1, readNum2;
28          inFile >> readNum1 >> readNum2;
29          inFile.close();
30
31          // Вычисление геометрического среднего
32          double geometricMean = std::sqrt(readNum1 * readNum2);
33
34          // Вывод чисел и результата на консоль
35          std::cout << "Исходные числа: " << readNum1 << " и " << readNum2 << std::endl;
36          std::cout << "Геометрическое среднее: " << geometricMean << std::endl;
37      } else {
38          std::cout << "Ошибка при открытии файла с данными." << std::endl;
39          return 1;
40      }
41
42      return 0;
43 }
```

Введите второе число: 2
Данные успешно сохранены в файл.
Исходные числа: 1 и 2
Геометрическое среднее: 1.41421

 Run  Debug  Stop  Share  Save

main.cpp data.txt ⋮

1 1 2

Лабораторная №9

```
#include <SFML/Graphics.hpp>
#include <iostream>

// Парменова Дарья Вариант 12
int main()
{
    // Создаем окно размером 800x600 пикселей
    sf::RenderWindow window(sf::VideoMode(600, 600), "Image Converter");

    // Открываем изображение из файла
    sf::Image image;
    if (!image.loadFromFile("/Users/apple/Desktop/flower.jpg"))
    {
        std::cerr << "Не получилось открыть изображение((" << std::endl;
        return 1;
    }

    // Получаем размеры изображения
    unsigned int width = image.getSize().x;
    unsigned int height = image.getSize().y;

    // Преобразуем изображение в серое с 256 градациями яркости
    for (unsigned int y = 0; y < height; ++y)
    {
        for (unsigned int x = 0; x < width; ++x)
        {
            sf::Color pixel = image.getPixel(x, y);
            // Вычисляем среднее значение каналов RGB
            uint8_t gray = (pixel.r + pixel.g + pixel.b) / 3;
            // Устанавливаем серое значение для каждого канала
            pixel.r = gray;
            pixel.g = gray;
            pixel.b = gray;
            // Устанавливаем пиксель с новым серым цветом
            image.setPixel(x, y, pixel);
        }
    }

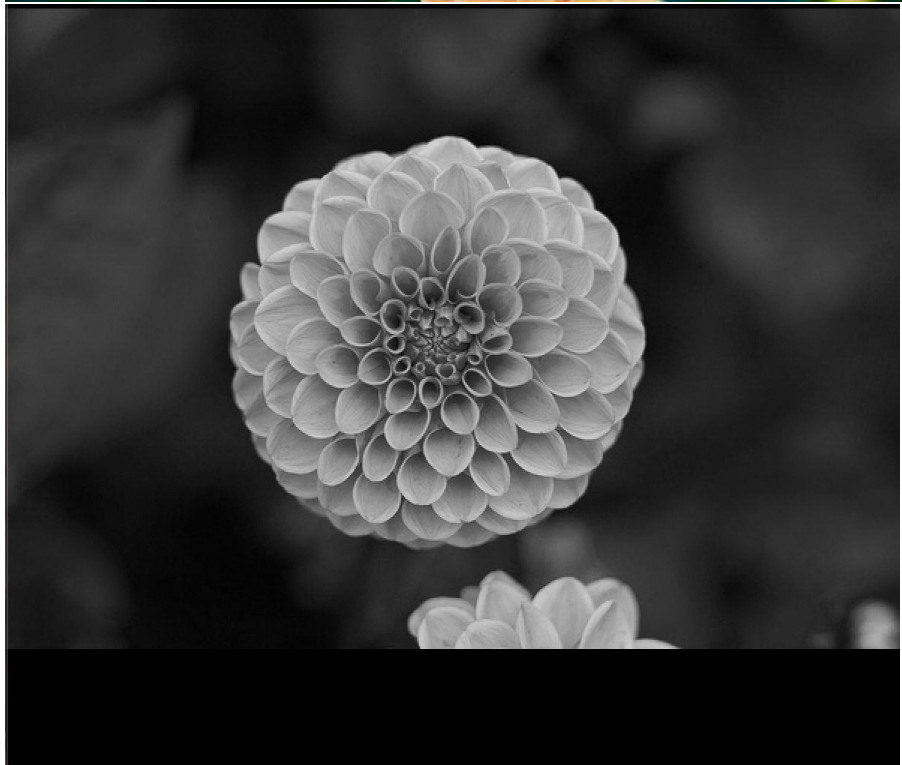
    // Создаем текстуру и спрайт для отображения изображения
    sf::Texture texture;
    texture.loadFromImage(image);
    sf::Sprite sprite(texture);

    // Выравниваем спрайт по центру окна
    sprite.setOrigin(sprite.getLocalBounds().width / 2.0f, sprite.getLocalBounds().height / 2.0f);
    sprite.setPosition(window.getSize().x / 2.0f, window.getSize().y / 2.0f);

    // Основной цикл приложения
    while (window.isOpen())
    {
        // Проверяем события окна
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        // Отрисовка спрайта на экране
        window.clear();
        window.draw(sprite);
        window.display();
    }

    return 0;
}
```



Лабораторная №10

```

#include <windows.h>
#include <cmath> // для функции exp

// Определение функции y(x)
double someFunction(double x) {
    return (4 * x * x + 2 * x) * exp(2 * x - 1);
}

// Глобальные переменные для размеров окна
const int WIDTH = 800;
const int HEIGHT = 600;

// Обработчик сообщений окна
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
        case WM_PAINT: {
            PAINTSTRUCT ps;
            HDC hdc = BeginPaint(hwnd, &ps);

            // Установка белого фона
            HBRUSH brush = CreateSolidBrush(RGB(255, 255, 255));
            FillRect(hdc, &ps.rcPaint, brush);
            DeleteObject(brush);

            // Установка черных осей
            HPEN pen = CreatePen(PS_SOLID, 2, RGB(0, 0, 0));
            SelectObject(hdc, pen);
            MoveToEx(hdc, 0, HEIGHT / 2, NULL);
            LineTo(hdc, WIDTH, HEIGHT / 2);
            MoveToEx(hdc, WIDTH / 2, 0, NULL);
            LineTo(hdc, WIDTH / 2, HEIGHT);
            DeleteObject(pen);

            // Отрисовка графика функции
            pen = CreatePen(PS_SOLID, 2, RGB(255, 0, 0));
            SelectObject(hdc, pen);

            double step = 0.01; // Шаг для x
            for (double x = -WIDTH / 200.0; x < WIDTH / 200.0; x += step) {
                double y = someFunction(x);
                int screenX = WIDTH / 2 + static_cast<int>(x * 100);
                int screenY = HEIGHT / 2 - static_cast<int>(y * 100);
                SetPixel(hdc, screenX, screenY, RGB(255, 0, 0));
            }

            DeleteObject(pen);
            EndPaint(hwnd, &ps);
            return 0;
        }
    }
}

```

```

        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;

        default:
            return DefWindowProc(hwnd, uMsg, wParam, lParam);
    }
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
    const char CLASS_NAME[] = "GraphWindowClass";

    WNDCLASS wc = {};
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    RegisterClass(&wc);

    HWND hwnd = CreateWindowEx(
        0,
        CLASS_NAME,
        "Graph Plotter",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT, WIDTH, HEIGHT,
        NULL,
        NULL,
        hInstance,
        NULL
    );

    if (hwnd == NULL) {
        return 0;
    }

    ShowWindow(hwnd, nCmdShow);

    MSG msg = {};
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
}

```